Python for Data Analysis: QuickStart

Griffin Golde

https://github.com/ggolde/python_workshop1_MBI

- The “homepage” of the workshop
  - All the links and files you need
  - Links to downloads for setups
    - Download the repository!

The Load-Out

VS Code – code editing

miniconda – package and environment management

Jupyter Notebooks – interactive block-by-block coding

homebrew – application manager (MacOS only, optional)

Next Steps

- Open command prompt / terminal and run:

```
conda create -n datasci python=3.12 numpy pandas scipy matplotlib
```

- Download the workshop repository
- Open VS Code
- File > Open Folder > workshop repository

caeser_paper_clean (Workspace)

EXPLORER

CAESER_PAPER_CLEAN (WORKSPACE)

- caeser_paper_clean
  - .vscode
  - data
  - figs
    - bla
    - comparison
    - immune
    - nac
    - umaps
    - style.mplstyle
  - results
  - scripts
    - clean_source_data.py
    - make_gene_sets_human.R
    - make_gene_sets_rat.R
  - _behavior_analysis.ipynb
  - _human_rna_analysis.ipynb
  - _rna_analysis.ipynb
  - _sc_atlas.ipynb
  - _visualization.ipynb
  - caeser_paper_clean.code-workspace
  - pyrightconfig.json
- pyRNASeqTools

OUTLINE

- bla_gc
- nac_gc
- gc
- smp_info
- meta
- bla_meta
- nac_meta

TIMELINE clean_source_data.py

- File Saved 8 mins
- File Saved 1 mo
- File Saved

_rna_analysis.ipynb | main.py M | clean_source_data.py

Open Files

Actions

Minimap

caeser_paper_clean > scripts > clean_source_data.py > ...

Editor

```python
import pandas as pd

bla_gc = pd.read_csv('../data/source/bla_gene_counts.csv', index_col=0).T
nac_gc = pd.read_csv('../data/source/nac_gene_counts.csv', index_col=0).T
bla_gc.index = bla_gc.index.astype('str') + '_bla'
nac_gc.index = nac_gc.index.astype('str') + '_nac'
gc = pd.concat([bla_gc, nac_gc])

smp_info = pd.read_csv('../data/source/sample_info.csv', index_col=0)
smp_info['behavioral_outlier'] = smp_info['behavioral_outlier'].convert_dtypes().fillna(False)
smp_info['health_concerns'] = smp_info['health_concerns'].fillna('None')

meta = pd.read_csv('../data/source/metadata.csv', index_col=0)
meta = meta.join(smp_info.filter(['behavioral_outlier', 'health_concerns']))

bla_meta = meta.copy()
nac_meta = meta.copy()

bla_meta.insert(0, 'tissue', 'bla')
nac_meta.insert(0, 'tissue', 'nac')

bla_meta.index = bla_meta.index.astype(str) + '_bla'
nac_meta.index = nac_meta.index.astype(str) + '_nac'
```

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS | JUPYTER

```
ggolde@mbp caeser_paper_clean %  conda activate rnaseq
(rnaseq) ggolde@mbp caeser_paper_clean % pwd
/Users/ggolde/home/bizon_lab/caeser_paper_clean
(rnaseq) ggolde@mbp caeser_paper_clean % echo "I am a terminal"
I am a terminal
(rnaseq) ggolde@mbp caeser_paper_clean %
```

zsh caeser...
zsh caeser...

CHAT

Build with Agent

AI responses may be inaccurate

Generate Agent Instructions to onboard AI onto your codebase.

clean_source_data.py

Describe what to build

Auto

Local | Default Approvals

master* | 0 0 | Ln 1, Col 1 | Spaces: 4 | UTF-8 | LF | Python | Finish Setup | rnaseq (3.12.12)

caeser_paper_clean (Workspace)

EXPLORER

CAESER_PAPER_CLEAN (WORKSPACE)

- caeser_paper_clean
  - .vscode
  - data
  - figs
    - bla
    - comparison
    - immune
    - nac
    - umaps
    - style.mplstyle
  - results
  - scripts
    - clean_source_data.py
    - make_gene_sets_human.R
    - make_gene_sets_rat.R
  - _behavior_analysis.ipynb
  - _human_rna_analysis.ipynb
  - _rna_analysis.ipynb
  - _sc_atlas.ipynb
  - _visualization.ipynb
  - caeser_paper_clean.code-workspace
  - pyrightconfig.json
- pyRNASeqTools

File Explorer

OUTLINE

- bla_gc
- nac_gc
- gc
- smp_info
- meta
- bla_meta
- nac_meta

Outline

TIMELINE clean_source_data.py

- File Saved 8 mins
- File Saved 1 mo
- File Saved

File history

_rna_analysis.ipynb | main.py M | clean_source_data.py

caeser_paper_clean > scripts > clean_source_data.py > ...

```python
import pandas as pd

bla_gc = pd.read_csv('../data/source/bla_gene_counts.csv', index_col=0).T
nac_gc = pd.read_csv('../data/source/nac_gene_counts.csv', index_col=0).T
bla_gc.index = bla_gc.index.astype('str') + '_bla'
nac_gc.index = nac_gc.index.astype('str') + '_nac'
gc = pd.concat([bla_gc, nac_gc])

smp_info = pd.read_csv('../data/source/sample_info.csv', index_col=0)
smp_info['behavioral_outlier'] = smp_info['behavioral_outlier'].convert_dtypes().fillna(False)
smp_info['health_concerns'] = smp_info['health_concerns'].fillna('None')

meta = pd.read_csv('../data/source/metadata.csv', index_col=0)
meta = meta.join(smp_info.filter(['behavioral_outlier', 'health_concerns']))

bla_meta = meta.copy()
nac_meta = meta.copy()

bla_meta.insert(0, 'tissue', 'bla')
nac_meta.insert(0, 'tissue', 'nac')

bla_meta.index = bla_meta.index.astype(str) + '_bla'
nac_meta.index = nac_meta.index.astype(str) + '_nac'
```

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS | JUPYTER

```
ggolde@mbp caeser_paper_clean %  conda activate rnaseq
(rnaseq) ggolde@mbp caeser_paper_clean % pwd
/Users/ggolde/home/bizon_lab/caeser_paper_clean
(rnaseq) ggolde@mbp caeser_paper_clean % echo "I am a terminal"
I am a terminal
(rnaseq) ggolde@mbp caeser_paper_clean %
```

zsh caeser...
zsh caeser...

CHAT

to close it

GitHub CoPilot

Build with Agent

AI responses may be inaccurate

Generate Agent Instructions to onboard AI onto your codebase.

clean_source_data.py

Describe what to build

Auto

Local | Default Approvals

master* | 0 0 | Ln 1, Col 1 | Spaces: 4 | UTF-8 | LF | Python | Finish Setup | rnaseq (3.12.12)

caeser_paper_clean (Workspace)

EXPLORER

CAESER_PAPER_CLEAN (WORKSPACE)

- caeser_paper_clean
  - .vscode
  - data
  - figs
    - bla
    - comparison
    - immune
    - nac
    - umaps
    - style.mplstyle
  - results
  - scripts
    - clean_source_data.py
    - make_gene_sets_human.R
    - make_gene_sets_rat.R
  - _behavior_analysis.ipynb
  - _human_rna_analysis.ipynb
  - _rna_analysis.ipynb
  - _sc_atlas.ipynb
  - _visualization.ipynb
  - caeser_paper_clean.code-workspace
  - pyrightconfig.json
- pyRNASeqTools

OUTLINE

- bla_gc
- nac_gc
- gc
- smp_info
- meta
- bla_meta
- nac_meta

TIMELINE clean_source_data.py

- File Saved 8 mins
- File Saved 1 mo
- File Saved

_rna_analysis.ipynb | main.py M | clean_source_data.py

caeser_paper_clean > scripts > clean_source_data.py > ...

```python
import pandas as pd

bla_gc = pd.read_csv('../data/source/bla_gene_counts.csv', index_col=0).T
nac_gc = pd.read_csv('../data/source/nac_gene_counts.csv', index_col=0).T
bla_gc.index = bla_gc.index.astype('str') + '_bla'
nac_gc.index = nac_gc.index.astype('str') + '_nac'
gc = pd.concat([bla_gc, nac_gc])

smp_info = pd.read_csv('../data/source/sample_info.csv', index_col=0)
smp_info['behavioral_outlier'] = smp_info['behavioral_outlier'].convert_dtypes().fillna(False)
smp_info['health_concerns'] = smp_info['health_concerns'].fillna('None')

meta = pd.read_csv('../data/source/metadata.csv', index_col=0)
meta = meta.join(smp_info.filter(['behavioral_outlier', 'health_concerns']))

bla_meta = meta.copy()
nac_meta = meta.copy()

bla_meta.insert(0, 'tissue', 'bla')
nac_meta.insert(0, 'tissue', 'nac')

bla_meta.index = bla_meta.index.astype(str) + '_bla'
nac_meta.index = nac_meta.index.astype(str) + '_nac'
```

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS | JUPYTER

```
ggolde@mbp caeser_paper_clean %  conda activate rnaseq
(rnaseq) ggolde@mbp caeser_paper_clean % pwd
/Users/ggolde/home/bizon_lab/caeser_paper_clean
(rnaseq) ggolde@mbp caeser_paper_clean % echo "I am a terminal"
I am a terminal
(rnaseq) ggolde@mbp caeser_paper_clean %
```

zsh caeser...
zsh caeser...

Manage multiple terminals

Terminal, CLI, and output

CHAT

Build with Agent

AI responses may be inaccurate

Generate Agent Instructions to onboard AI onto your codebase.

clean_source_data.py

Describe what to build

Auto

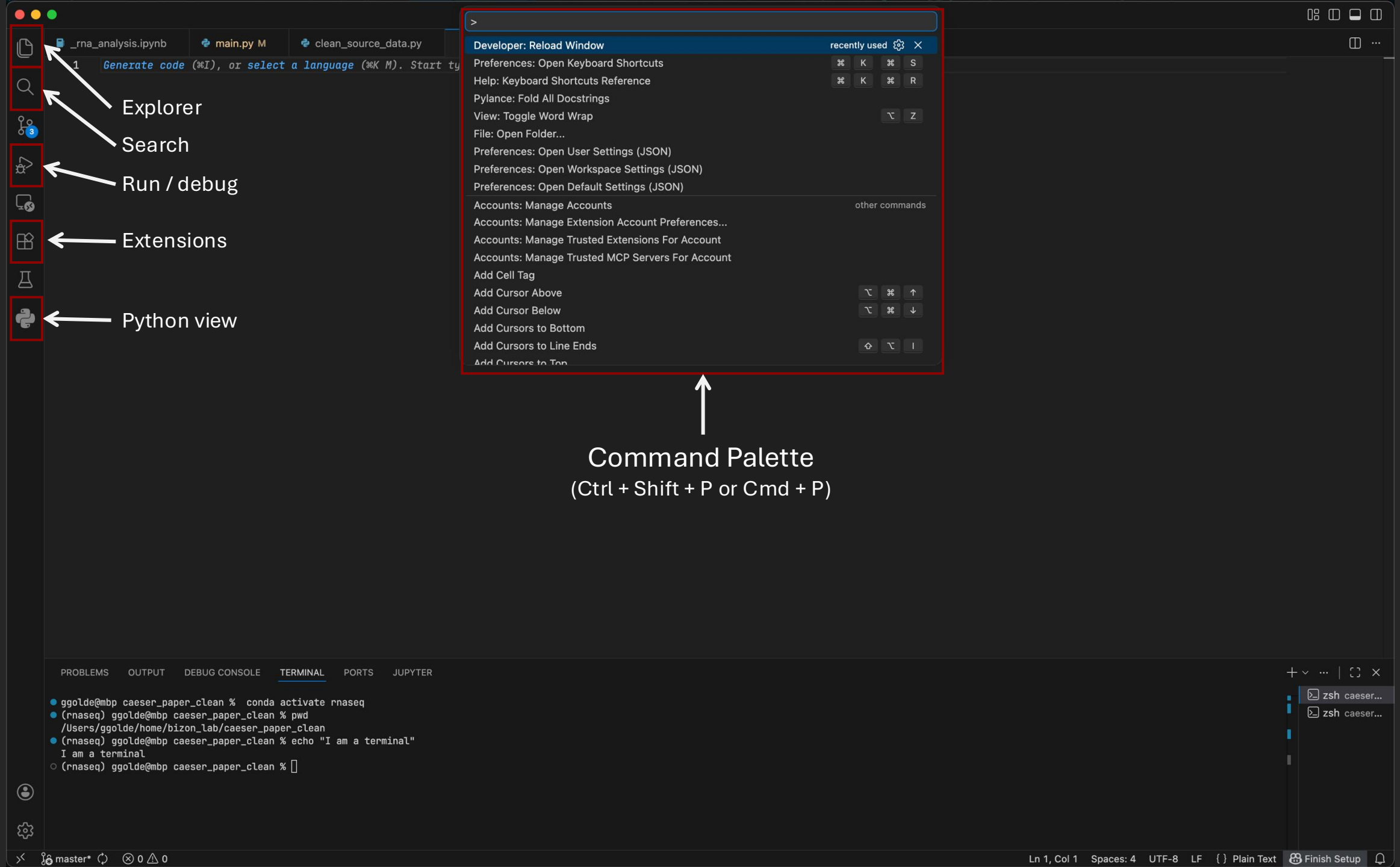
Local | Default Approvals

master* | 0 0

Ln 1, Col 1 | Spaces: 4 | UTF-8 | LF | Python | Finish Setup | rnaseq (3.12.12)

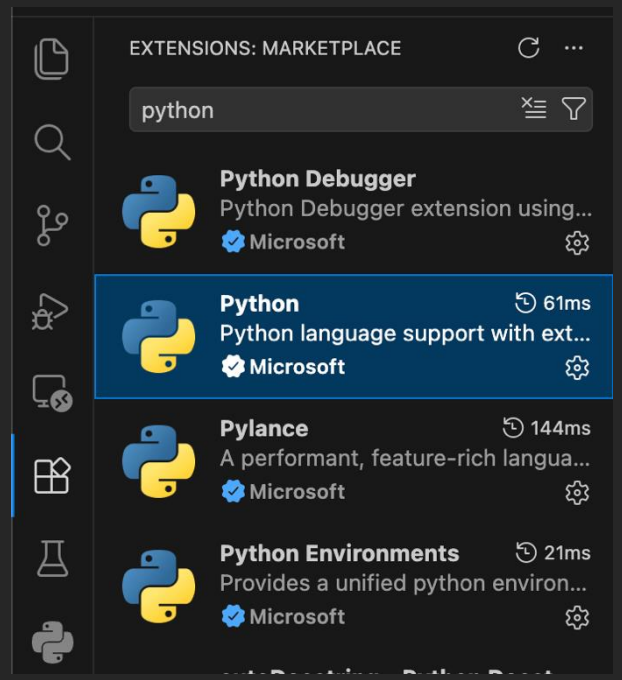
See current:

- code language
- type encoding
- environment
- notifications

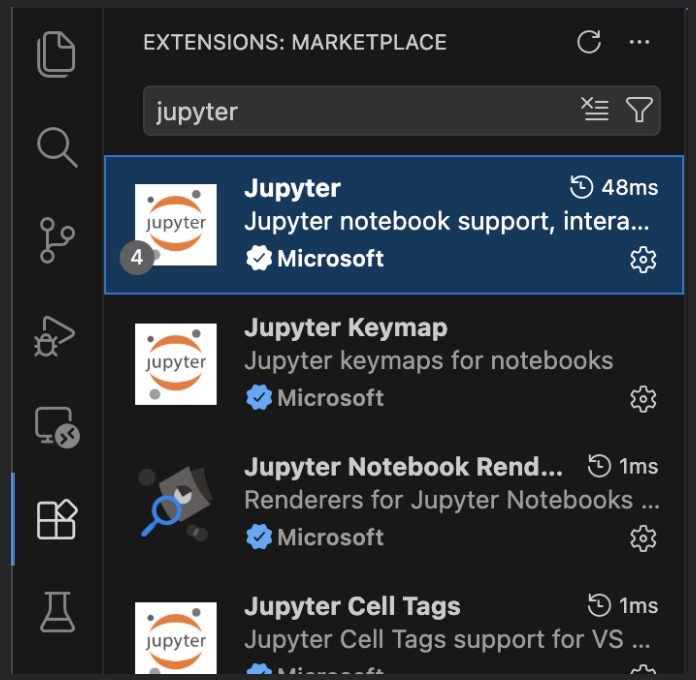

_rna_analysis.ipynb
main.py M
clean_source_data.py
Generate code (⌘I), or select a language (⌘K M). Start ty
Explorer
Search
Run / debug
Extensions
Python view
>
Developer: Reload Window
recently used
Preferences: Open Keyboard Shortcuts
Help: Keyboard Shortcuts Reference
Pylance: Fold All Docstrings
View: Toggle Word Wrap
File: Open Folder...
Preferences: Open User Settings (JSON)
Preferences: Open Workspace Settings (JSON)
Preferences: Open Default Settings (JSON)
Accounts: Manage Accounts
other commands
Accounts: Manage Extension Account Preferences...
Accounts: Manage Trusted Extensions For Account
Accounts: Manage Trusted MCP Servers For Account
Add Cell Tag
Add Cursor Above
Add Cursor Below
Add Cursors to Bottom
Add Cursors to Line Ends
Command Palette
(Ctrl + Shift + P or Cmd + P)
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
JUPYTER
ggolde@mbp caeser_paper_clean % conda activate rnaseq
(rnaseq) ggolde@mbp caeser_paper_clean % pwd
/Users/ggolde/home/bizon_lab/caeser_paper_clean
(rnaseq) ggolde@mbp caeser_paper_clean % echo "I am a terminal"
I am a terminal
(rnaseq) ggolde@mbp caeser_paper_clean %
zsh caeser...
zsh caeser...
master*
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Plain Text
Finish Setup

Go to extensions and install:

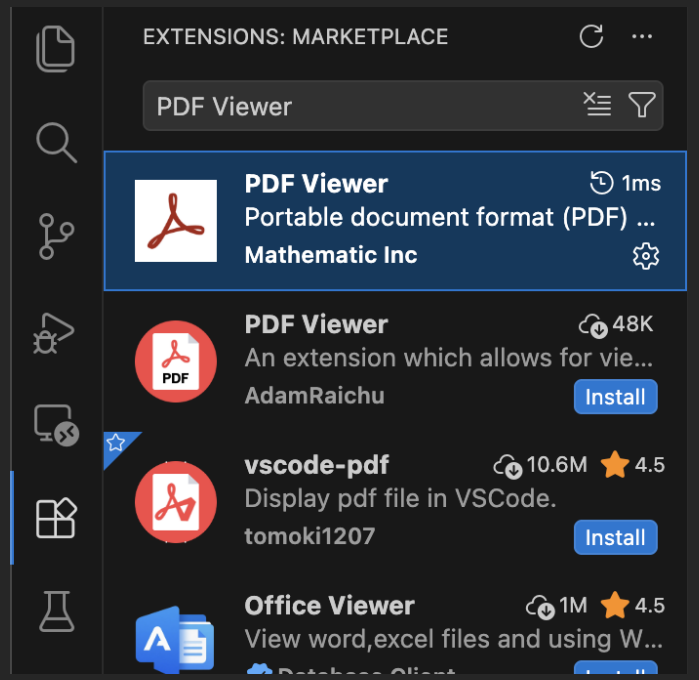
Python Extensions



Jupyter Notebook



PDF Viewer (optional)



workshop1

EXPLORER

WORKSHOP1
- data
- resources
  - numpy_cheat_sheet.pdf
  - pandas_cheat_sheet.pdf
  - python_cheat_sheet.pdf
- excercises.ipynb

2. Open exercises.ipynb

excercises.ipynb > Section 0 - Jupyter > # example code: run this cell

Select Kernel

3. Select kernel
conda > datasci

Python Workshop: Quick Start

Section 0 - Jupyter

Jupyter Notebook cells are either: Markdown (text with some formatting) or code which you can run.
Markdown heading are show and can be navigated in the Outline section of the last panel.

1. Select the datasci enviroment as the kernal
2. Run the cell below with the play button or Ctrl + Enter

```python
# example code: run this cell
print('Hello World!')
```

[2] Python

```
Hello World!
```

1. Download the repo and open it in VS Code

Section 1 - Python

Exercise 1.1

Topics: variables and data types

Create the variables:

- x with value 5
- y with value 7.6
- z with value "John"

Use the type() function to print the types of each. The type function returns the class of the value put into it.

```python
# example: type() function
mylist = ['John', 'James', 'Jessie', 'Jonathan']
print(type(mylist))
```

[3] Python

```
<class 'list'>
```

```python
# solution
```

OUTLINE
- Python Workshop: Quick Start
- Section 0 - Jupyter
- Section 1 - Python
  - Exercise 1.1
  - Exercise 1.2
  - Exercise 1.3
  - Exercise 1.4
  - Exercise 1.5

TIMELINE

1 0 Update is ready, click to restart. Spaces: 4 Cell 4 of 23 Select Kernel

workshop1

EXPLORER

WORKSHOP1
- data
- resources
  - numpy_cheat_sheet.pdf
  - pandas_cheat_sheet.pdf
  - python_cheat_sheet.pdf
- excercises.ipynb

excercises.ipynb > Section 0 - Jupyter > # example code: run this cell

Select Kernel

Python Workshop: Quick Start

Section 0 - Jupyter

Jupyter Notebook cells are either: Markdown (text with some formatting) or code which you can run. Markdown heading are show and can be navigated in the Outline section of the last panel.

1. Select the datasci enviroment as the kernal
2. Run the cell below with the play button or Ctrl + Enter

```
# example code: run this cell
print('Hello World!')
```

[2] Python

```
Hello World!
```

Section 1 - Python

Exercise 1.1

Topics: variables and data types

Create the variables:

- x with value 5
- y with value 7.6
- z with value "John"

Use the type() function to print the types of each. The type function returns the class of the value put into it.

```
# example: type() function
mylist = ['John', 'James', 'Jessie', 'Jonathan']
print(type(mylist))
```

[3] Python

```
<class 'list'>
```

```
# solution
```

Markdown (text) cells
- Double click to edit
- Ctrl + Enter to format

Code cell & output
- Ctrl + Enter to run

Notebook Controls
- Run all, run below, run above, split cell, etc.

Outline
- Navigate markdown headers

OUTLINE
- Python Workshop: Quick Start
- Section 0 - Jupyter
- Section 1 - Python
  - Exercise 1.1
  - Exercise 1.2
  - Exercise 1.3
  - Exercise 1.4
  - Exercise 1.5

TIMELINE

1 0 Update is ready, click to restart.

Spaces: 4 { } Cell 4 of 23 Select Kernel

Variables & data types

```
# numeric
int_ = 634
float_ = 5.0
complex_ = 3 + 7j
bool_ = True

# strings
str_ = "hello”

# none type
none_ = None
```

```
# containers
list_ = [0, 1, "string", False]
set_ = {0, 0, "will de-duplicate"}
tuple_ = (7, 11, 13)
dict_ = {
    "key0" : "value0",
    64 : 8,
    49 : 7,
}
```

Variable naming

```
# valid
myvar = "John"
my_var = "John"
_my_var = "John"
myVar = "John"
MYVAR = "John"
myvar2 = "John"
```

```
# invalid
2myvar = "John"
my-var = "John"
my var = "John"
```

```
# malpractice
int = 90
bool = False
str = "bad"
range = 5.5
```

```
# type hinting
i_am_float: float = 4.2
```

Operators

```
i = 8
j = 3

res0 = i + j # 11
res1 = i - j # 5
res2 = i * j # 24
res3: float = i / j # 2.66...

res4 = i ** j # 512
res5: int = i % j # 2
res6: int = i // j # 2

eq = (i + j) / (j - 1) # 5.5
```

```
i = 4
# assignment + operation
i += 1 # 5 (i = i + 1)
i -= 1 # 4
i *= 2 # 8
i **= 2 # 64
i //= 2 # 32
i /= 8 # 4.0
i %= 3 # 1.0

# boolean math
cond0 = True & False # False
cond1 = True | False # True
```

Type casting

```
x = int(1) # 1
y = int(2.8) # 2
z = int("3") # 3
```

```
x = float(1) # 1.0
y = float(2.8) # 2.8
z = float("3") # 3.0
w = float("4.2") # 4.2
```

Strings

```
ex = """Sample text
with multiple lines
and lots of words"""

# strings are iterables
a = "Hello, World!"
first = a[0] # "H"
length: int = len(a) # 13

# strings can be added
a = "Go" + " " + "Gators" + "!"
print(a)
```

Go Gators!

```
# f-strings
i: float = 1 / 3
j: float = 1e-6

f_str = f"i = {i:.2f}, j = {j:.2e}"
print('result:', f_str)
```

result: i = 0.33, j = 1.00e-06

Lists

```
var = ["oak", "elm", "pine"]

# accessing & assigning
first = var[0] # oak
second = var[1] # elm
last = var[-1] # pine
var[2] = "spruce" # ["oak", "elm", "spruce"]

# adding values
var.append("fir") # to end
var.insert(1, "cherry") # to second position

# slicing
my_slice = var[1:3] # from index 1 to before 3
```

```
print(var)
print(my_slice)
```

```
['oak', 'cherry', 'elm', 'spruce', 'fir']
['cherry', 'elm']
```

[‘lists’, ‘tuples’, ‘sets’]

	list	tuple	set
syntax	[...]	(...)	{...}
constructor	list()	tuple()	set()
duplicate items?	Yes	Yes	No
muatable?	Yes	No	Yes
ordered?	Yes	Yes	No

```
# conversions
my_list = [1, 1, 2, 3]
my_tuple = tuple(my_list) # (1, 1, 2, 3)
my_set = set(my_list) # {1, 2, 3}
```

```
# tuple immutability example
my_tuple = (7, 11, 13)
my_tuple[0] = 1 # will error
```

Dictionaries

```
my_dict = {
        'key0' : 'I will be replaced',
        'key0' : 'new value',
        100 : 'mapped',
        'dupe' : 'mapped',
        'remove_me' : 10,
}

# access
val0 = my_dict[100]
val1 = my_dict['key0']
```

```
# add / remove
my_dict['key1'] = 4.2
removed = my_dict.pop('remove_me')

print(removed)
print(my_dict)
```

10

{'key0': 'new value', 100: 'mapped', 'dupe': 'mapped', 'key1': 4.2}

Copy vs. Reference

```python
# reference
my_list = [0, 1, 2, 3]
ref_list: list = my_list

ref_list[0] = 'zero'
print(my_list)
```

['zero', 1, 2, 3]

```python
# copy
my_list = [0, 1, 2, 3]
copy_list = my_list.copy()
copy_list2 = my_list[:]

copy_list[0] = 'zero'
print(my_list)
```

[0, 1, 2, 3]

```python
# dicts are similar
my_dict = {'key', 'value'}
copy_dict = my_dict.copy()
```

If else & comparators

```
score = 75

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"

print('Grade:', grade)
```

Grade: C

```
i = 3
i == 5 # False
i != 5 # True

i > 5 # False
i < 5 # True
i >= 5 # False
i <= 5 # True

sentence = "Stronger together Vis"
subset = "ong”

subset in sentence # True
"vis" not in sentence # True
subset == "ong" # True
```

Match (less common)

```
day = 4
match day:
    case 1:
        print("Monday")
    case 2:
        print("Tuesday")
    case 3:
        print("Wednesday")
    case 4:
        print("Thursday")
    case 5:
        print("Friday")
    case 6:
        print("Saturday")
    case 7:
        print("Sunday")
    case _: # default case
        print("not a day")
```

Thursday

Loops

```python
fruits = ["apple", "grape", "banana", "pineapple", "cherry", "kiwi"]

# for loop
for x in fruits:
    if "apple" in x:
        continue # passes to the next iteration
    if x[-1] == "y":
        break # breaks out of the loop
    print(x)
```

grape

banana

```python
num = 14
i = 1

# while loop
while i < num:
    if i > (num / 2):
        break
    elif num % i == 0:
        print(i)
    i += 1
```

1 2 7

Functions

```python
def basic_function():
    print("Journey before destination")

def calculator(arg1, arg2, how: str = 'add') -> float:
    if how == 'add':
        return arg1 + arg2
    elif how == 'subtract':
        return arg1 - arg2
    elif how == 'multiply':
        return arg1 * arg2
    elif how == 'divide':
        return arg1 / arg2
    else:
        raise ValueError(f'{how} not recognized')
```

```python
basic_function()
res0 = calculator(3, arg2 = 5, how='multiply')
res1 = calculator(170, 5.0)
print(res0, res1)
```

Journey before destination

15 175.0

Classes

```python
class Date:
    def __init__(self, weekday, day, month, year) -> None:
        self.month_names = [
            'January', 'February', 'March', 'April', 'May', 'June',
            'July', 'August', 'September', 'October', 'November', 'December'
        ]

        self.weekday = weekday
        self.day = day
        self.month = month
        self.year = year

        if month not in self.month_names:
            raise Warning(f'{month} not a recognized month.')

    def get_month_num(self) -> int:
        mapping = dict(zip(self.month_names, range(1, len(self.month_names), 1)))
        return mapping[self.month]
```

Classes...

```
class Date:
        ...
        def print_pretty_date(self, ith: str = 'th') -> None:
    pretty_date = f'{self.weekday}, {self.month} {self.day}{ith}, {self.year}'
    print(pretty_date)

  def get_numeric_date(self) -> str:
    month_num = self.get_month_num()
    num_date = f'{month_num}/{self.day}/{self.year}'
    return num_date

today = Date('Friday', 27, 'March', 2026)
today.print_pretty_date(ith='th')
print(today.get_numeric_date())
```

Friday, March 27th, 2026

3/27/2026

```
# accessing class vars
print(today.weekday, today.day)
```

Friday 27

List comprehension (compact for loops)

```
newlist = [expression for item in iterable if condition]
newlist = [expression if condition else expression for item in iterable]

fruits = ["apple", "grape", "banana", "pineapple", "cherry", "kiwi"]
# long form
newlist = []
for x in fruits:
    if "apple" not in x:
    newlist.append(x[0].upper() + x[1:])

# short form
newlist = [x[0].upper() + x[1:] for x in fruits if 'apple' not in x]
```

```
print(newlist)
['Grape', 'Banana', 'Cherry', 'Kiwi']
```

Exercises 1 - Python

- Do the exercises in section 1 of exercises.ipynb
- Work together and ask me for help
- Reference the slides, cheat sheets, and W3schools

Numpy Arrays

```python
import numpy as np

arr = np.array([5, 9, 8, 3, 2])
arr.shape # (5,)
arr.dtype # int64

# access with indexing
arr[0] # 5

# operations
arr.sum() # 27
arr.mean() # 5.4
arr.std() # 2.72...
```

Arrays are like lists but:

- can only contain one type
- multiple dimension support
- easy mathematical operations

```python
# broadcasting
arr + 5 # [10, 14, 13, 8, 7]
arr * 2 # [10, 18, 16, 6, 4]
```

Numpy Arrays...

```
arr = np.array(
        [[7, 3, 9],
         [3, 1, 3],
         [5, 8, 0],
         [6, 4, 2]]
)
# shape
arr.shape # (4, 3)

# indexing
arr[1, 2] # 3
arr[-1, 1:] # [4, 2]
```

```
# can operate along axes
arr.sum() # 51
arr.sum(axis=0) # [21, 16, 14] (3,)
arr.sum(axis=1) # [19, 7, 13, 12] (4,)
arr.sum(axis=0, keepdims=True) # [[21, 16, 14]] (1,3)

# broadcasting
add2 = arr + 2
add2 = [[9, 5, 11],
        [5, 3, 5 ],
        [7, 10, 2],
        [8, 6, 4 ]]
```

Boolean indexing

```
arr = np.array(
        [[ 8, 10, 1, 16, 19],
         [ 8, 8, 19, 7, 11],
         [ 5, 6, 2, 19, 6]]
)
# boolean indexing
mask = arr > 10
mask = np.array(
        [[False, False, False, True, True],
         [False, False, True, False, True],
         [False, False, False, True, False]]
)
```

```
arr[arr > 10] # [16, 19, 19, 11, 19]
arr[arr > 10] = 10 # cap values
print(arr)
```

```
array([[ 8, 10,  1, 10, 10],
       [ 8,  8, 10,  7, 10],
       [ 5,  6,  2, 10,  6]])
```

Broadcasting

(4, 3)

[[7, 3, 9],
[3, 1, 3],
[5, 8, 0],
[6, 4, 2]]

×

(1, 3)

[[0, 1, 0]]

→

(4, 3)

[[0, 3, 0],
[0, 1, 0],
[0, 8, 0],
[0, 4, 0]]

[[7, 3, 9],
[3, 1, 3],
[5, 8, 0],
[6, 4, 2]]

(1 x 4, 3)

[[0, 1, 0],
[0, 1, 0],
[0, 1, 0],
[0, 1, 0]]

Broadcasting

(4, 3)

[[7, 3, 9],
[3, 1, 3],
[5, 8, 0],
[6, 4, 2]]

×

(4, 1)

[[0],
[1],
[0],
[1]]

→

(4, 3)

[[0, 0, 0],
[3, 1, 3],
[0, 0, 0],
[6, 4, 2]]

[[7, 3, 9],
[3, 1, 3],
[5, 8, 0],
[6, 4, 2]]

×

(4, 1 x 3)

[[0, 0, 0],
[1, 1, 1],
[0, 0, 0],
[1, 1, 1]]

↗

Manipulating Shape

```
arr = np.array(
        [[7, 3, 9],
         [3, 1, 3],
         [5, 8, 0],
         [6, 4, 2]]
)

# flatten
arr.ravel()
```

```
[7, 3, 9, 3, 1, 3, 5, 8, 0, 6, 4, 2]
```

```
# reshape
arr.reshape((6, 2))
```

```
[[7, 3],
 [9, 3],
 [1, 3],
 [5, 8],
 [0, 6],
 [4, 2]]
```

```
# resize (inplace)
arr.resize((3, 2))
```

```
[[7, 3],
 [9, 3],
 [1, 3]]
```

Manipulating Data

```
arr = np.array(
        [[7, 3, 9],
         [3, 1, 3],
         [5, 8, 0],
         [6, 4, 2]]
)

# normalize rows by mean
mean = arr.mean(axis=1)
res = arr / mean
```

ValueError: operands could not be broadcast together with shapes (4,3) (4,)

```
# normalize rows by mean
mean: np.ndarray = arr.mean(axis=1)
mean = mean.reshape((4, 1))
res = arr / mean

# --- OR ---
mean = arr.mean(axis=1, keepdims=True)
res = arr / mean

print(res.round(1))
```

```
[[1.1, 0.5, 1.4],
 [1.3, 0.4, 1.3],
 [1.2, 1.8, 0. ],
 [1.5, 1. , 0.5]]
```

Operations

```
a = np.array([6, 4, 1])
b = np.array([-4, 7, -2])

# combination
vert = np.vstack([a, b]) # [[6, 4, 1], [-4, 7, -2]]
horz = np.hstack([a, b]) # [6, 4, 1, -4, 7, -2]
comb = np.concatenate([a, b], axis=0) # [6, 4, 1, -4, 7, -2]

# operations accept axis and keepdims (usually)
arr = vert
arr.mean(axis=1, keepdims=True) # [[3.66..., 0.33...]]
arr.min(axis=1) # [1, -4]
arr.max(axis=1) # [6, 7]
arr.std(axis=1) # [2.05, 4.78]
np.median(arr, axis=1, keepdims=ToF) # [4, -2]
```

```
# numpy operations
np.add(a, b) # a + b = [2, 11, -1]
np.subtract(a, b) # a - b = [10, -3, 3]
np.multiply(a, b) # a * b = [-24, 28, -2]
np.divide(a, b) # a / b = [-1.5, 0.57, -0.5]

# numpy math
np.pi # 3.14...
np.e # 2.718...
np.log(a) # [1.79, 1.39, 0]
np.exp(a) # [403, 54.6, 2.71]
np.pow(a, 2) # [36, 16, 1]
np.sqrt(a) # [2.45, 2, 1]
np.sin(a) # [-0.28, -0.76, 0.84]
np.cos(a) # [0.96, -0.65, 0.54]
```

Exercises 2 - Numpy

- Do the exercises in section 2 of exercises.ipynb
- Work together and ask me for help
- Reference the slides, the numpy_cheat_sheet.pdf, and W3schools

Pandas Series

Index

Name

Values
(can be mixed types)

	Weight
Name	
Avery Bradley	180.0
Jae Crowder	235.0
Sergey Karasev	208.0
Sean Kilpatrick	219.0
Chris Paul	175.0
Paul Pierce	235.0

```python
import pandas as pd
series = pd.Series()

# attributes
series.index # ['Avery Bradley', 'Jae Crowder', ...]
series.size # 6
series.name # 'Weight'
series.dtype # float64

# access
series['Chris Paul'] # by index, 175.0
series.loc['Chris Paul'] # same as above
series.iloc[0] # by position index, 180.0

# aggregate
series.mean() # 208.66...
series.max() # 235.0

# useful functions
series.nunique() # 5
series.unique() # [180, 235, 208, 219, 175]

# boolean mask and filter
series = series[series > 200]
```

Pandas DataFrames

Index

Columns

	Team	Number	Position	Age	Height	Weight	College	Salary
Name								
Avery Bradley	Boston Celtics	0.0	PG	25.0	6-2	180.0	Texas	7730337.0
Jae Crowder	Boston Celtics	99.0	SF	25.0	6-6	235.0	Marquette	6796117.0
Sergey Karasev	Brooklyn Nets	10.0	SG	22.0	6-7	208.0	NaN	1599840.0
Sean Kilpatrick	Brooklyn Nets	6.0	SG	26.0	6-4	219.0	Cincinnati	134215.0
Chris Paul	Los Angeles Clippers	3.0	PG	31.0	6-0	175.0	Wake Forest	21468695.0
Paul Pierce	Los Angeles Clippers	34.0	SF	38.0	6-7	235.0	Kansas	3376000.0

Columns can contain values of mixed types

Pandas DataFrames...

```
# attributes
df.index # ['Avery Bradley', 'Jae Crowder', ...]
df.columns # ['Team', 'Number', 'Position'...]
df.shape # (458,8)

# access columns
df['Height'] # pd.Series of Hieght with index conserved
df[['Height']] # pd.DataFrame of shape (458, 1)

# access index
df.loc['Jae Crowder'] # pd.Series with columns now as index
df.iloc[1] # same as above

# access both
df.loc['Jae Crowder', 'College'] # 'Marquette'
```

Panda Power

```
# boolean index
df.loc[df['Team'] == 'Utah Jazz'] # only Utah Jazz players

# find average age of point gaurd
df.loc[df['Position'] == 'PG', 'Age'].mean() # 26.85

# better, find average age and pay of every position
df.groupby('Position').agg(
        age=('Age', 'mean'),
        salary=('Salary', 'mean'),
).round(2)

# what about the top 5 most represented colleges
df['College'].value_counts(sort=True).head(5)
```

Position	age	salary
C	27.37	5967052.00
PF	27.16	4562482.99
PG	26.85	5077829.22
SF	26.86	4857392.57
SG	26.54	4009861.19

College	count
Kentucky	22
Duke	20
Kansas	18
North Carolina	16
UCLA	15

Section 3 – Pandas Walkthrough

- Pandas is not the most intuitive package
- You will learn functions and methods as you need them
- We will walkthrough examples line by line before getting to the exercises

Section 4 – Applied Exercises

- Now we will tackle real-world data analysis problems
- First, we will fit a behavioral model to delayed discounting task data
- Second, we will process and analyze the response rates in a risk-decision making task